

Property availability & bookings - design (Phase 1)

Product: Kama Properties / isisel.com

Scope: Schema + REST API (host blocks, list bookings)

Date: May 2026

1. Goals

```
| Goal | Detail |
|-----|-----|
| One calendar per property | Every listing has availability data in MongoDB |
| Host blocks | Owner marks dates unavailable (maintenance, personal use) |
| Booking blocks | Confirmed stays auto-block dates (separate `Booking` collection) |
| APIs | Read merged availability; host update blocks; host list bookings |
| Next phase (not in v1 API UI) | Host calendar page from My listings -> Calendar |
```

2. Date rules (industry standard)

Stay: checkIn = 2026-06-10, checkOut = 2026-06-15

Blocked nights: 10, 11, 12, 13, 14 (checkOut day is departure, not a booked night)

3. Data model

3.1 `PropertyAvailability` (1 document per property)

```
| Field | Type | Description |
|-----|-----|-----|
| `propertyId` | ObjectId | Unique ref to Property |
| `defaultAvailability` | `open` \| `closed` | Default for dates with no block (default: `open`) |
| `hostBlocks` | Array | Manual host unavailability |
| `hostBlocks[].startDate` | String | `YYYY-MM-DD` |
| `hostBlocks[].endDate` | String | Exclusive end |
| `hostBlocks[].note` | String? | Optional, host-only |
| `createdAt` / `updatedAt` | Date | Timestamps |
```

Created when: property is listed (POST `/api/properties`) or lazily on first calendar access.

3.2 `Booking`

```
| Field | Type | Description |
|-----|-----|-----|
| `propertyId` | ObjectId | Property |
| `guestId` | String | NextAuth user id |
| `guestName` | String? | Snapshot |
| `guestEmail` | String? | Snapshot |
| `checkIn` | String | `YYYY-MM-DD` |
| `checkOut` | String | Exclusive end |
| `status` | Enum | `pending`, `confirmed`, `cancelled` |
| `transactionId` | Number? | Link to Flutterwave transaction |
| `createdAt` / `updatedAt` | Date | Timestamps |
```

Calendar impact: Only `confirmed` bookings block guest-facing dates. `pending` optional later for payment timeout.

4. API design

4.1 `GET /api/properties/[id]/availability`

Auth: Public (optional session for host detail)

Response (guest / public):

```
```json
{
```

```

"propertyId": "664a?",
"defaultAvailability": "open",
"unavailableRanges": [
 { "startDate": "2026-06-10", "endDate": "2026-06-15", "source": "booking" },
 { "startDate": "2026-07-01", "endDate": "2026-07-05", "source": "host" }
]
}
...

```

**\*\*Response** (property owner, same URL + session): **\*\* adds `hostBlocks`, `bookingsSummary` count.**

**\*\*Flow:**

...

Client API MongoDB

```

| | |
|-- GET /availability -->| |
| |-- find Property ----->|
| |-- find PropertyAvailability->|
| |-- find Booking (confirmed)->|
| |-- merge ranges ----->|
|<-- JSON -----| |
...

```

---

### 4.2 `PUT /api/properties/[id]/availability`

**\*\*Auth:** Verified host + `property.owner === session.user.id`

**\*\*Body:**

```

```json
{
  "defaultAvailability": "open",
  "hostBlocks": [
    { "startDate": "2026-07-01", "endDate": "2026-07-05", "note": "Renovation" }
  ]
}
...

```

****Validation:**

- Valid date format; `startDate < endDate`
- No overlap between host blocks
- No host block overlaps any ****confirmed**** booking
- Max block span (e.g. 365 days per block)

****Response:** Updated `hostBlocks` + `defaultAvailability`

****Flow:**

...

Host (My listings) API DB

```

| | |
|-- PUT hostBlocks ---->| |
| |-- assert owner ----->|
| |-- load bookings ----->|
| |-- validate vs bookings -->|
| |-- upsert Availability --->|
|<-- 200 OK -----| |
...

```

4.3 `GET /api/properties/[id]/bookings`

****Auth:** Property owner only

****Query:** `?status=confirmed` (optional filter)

****Response:**

```json

{

```

"propertyId": "664a?",
"bookings": [
{
 "_id": "?",
 "checkIn": "2026-06-10",
 "checkOut": "2026-06-15",
 "status": "confirmed",
 "guestName": "Jane Doe",
 "guestEmail": "jane@example.com",
 "createdAt": "2026-05-01T12:00:00.000Z"
}
]
}
...

```

## 5. Authorization matrix

| Endpoint                             | Guest | Signed-in user | Property owner | Admin |
|--------------------------------------|-------|----------------|----------------|-------|
| GET availability (public fields)     | ?     | ?              | ?              | ?     |
| GET availability (hostBlocks detail) | ?     | ?              | ?              | ?*    |
| PUT availability                     | ?     | ?              | ?              | ?*    |
| GET bookings                         | ?     | ?              | ?              | ?*    |

\\*Admin not required in Phase 1.

---

## 6. Integration points (later)

```

Payment success (Transaction POST)
 |
 v
Create/update Booking (confirmed)
 |
 v
Dates appear in GET availability
```

## 7. Phase 2 (UI - out of scope for this PDF/API-only delivery)

1. **My listings** card -> button **Calendar** -> `/properties/[id]/calendar``
  2. Month UI: tap to add/remove host blocks
  3. **Property page** check-in/out pickers consume ``GET availability``
  4. **Reserve** validates range before Flutterwave
- 

## 8. File map (implementation)

| File                                                         | Role                         |
|--------------------------------------------------------------|------------------------------|
| <code>`models/PropertyAvailability.js`</code>                | Host blocks + default        |
| <code>`models/Booking.js`</code>                             | Stays / booking records      |
| <code>`utils/availability/dateUtils.js`</code>               | Parse, compare, merge ranges |
| <code>`utils/availability/availabilityService.js`</code>     | Load, merge, validate        |
| <code>`utils/availability/propertyAccess.js`</code>          | Owner checks                 |
| <code>`app/api/properties/[id]/availability/route.js`</code> | GET, PUT                     |
| <code>`app/api/properties/[id]/bookings/route.js`</code>     | GET                          |

---

\*End of Phase 1 design document\*